

Programming SQL Server 2019

Writing Scripts and Batches

October 17, 2019 12:39 PM

Prelude

Every day in every session you are writing SQL scripts. Generally, scripts are saved in files. We will explore more details about various aspects of SQL scripts – using local and global variables, setting values in variables. We will learn SQL batches in depth. We will also learn how to execute dynamic SQL. Today you will be get introduced to new tool SQLCMD.

Go ahead and get started.

Today's topics

- SQL Scripts and Batches
- Variables in T-SQL script
- Setting values to variables
- System functions
- SQL batches
- Errors in SQL batches
- Working with SQLCMD utility
- Dynamic SQL with EXEC command
- Workouts
- Practice works

Home works

- Write a simple script that creates two integer variables (one called Var1 and one called Var2), places the values 2 and 4 in them respectively, and then outputs the value of the two variables added together.
- Create a variable called MinOrder and populate it with the smallest line item amount after discount for the Northwind CustomerNo 'ALFKI'. Output the final value of MinOrder.
- Use SQLCMD to output the results of the query `SELECT COUNT (*) FROM Customers` to the console window.

Important points

-
-
-

Summary

-
-
-

SQL Scripts

- A SQL script is a set of SQL commands saved as a file
- A SQL script can contain one or more T-SQL statements or T-SQL blocks.
- You can use SQL Scripts to create, edit, view, run, and delete objects.
- The file can be used as input to SQL Server Management Studio Code editor or the SQLCMD utilities
- Scripts generally have a unified goal.
- Scripts are stored as text files.
- SQL Server Management Studio Stores Scripts with .sql extension.
- SQL Server Management Studio helps you writing, testing scripts

Uses SQL Scripts

- Transact-SQL scripts can be used to do the following:
 - Keep a permanent copy of the steps used to create and populate the databases on your server, as a backup mechanism.
 - Transfer the statements from one computer to another when it is required.
 - Quickly educate new employees by enabling them to find problems in the code, understand the code, or change the code.

The USE statement

- The USE statement sets the current database.
- Syntax
USE database-name
- Statements that appear after the USE statement are executed in the specified database.

Comments in T-SQL

- Comment are text in SQL statements that are not executed.
- Programmers, Script authors are always encouraged to add comments – for later reference, others to understand.
- In T-SQL statements you can add comments in two ways

Single-line comment

- Starts with --
--Comment
- This is single line comment
- Text stating from the -- to the end of the line is treated as comments
- This type of comments can be inserted on a separate line, at the end of a Transact-SQL command line, or within a Transact-SQL statement
- Examples
--Insert data
Exec sp_addrow 1, --id,
'Habib;' --name

Multi-line comment

- Enclosed within /* and */
- /*.. */ comment
- This is multiline comment.
- All codes within /* and */ ignored
- Example
- The following example uses comments to explain what the section of the code is supposed to do.
USE AdventureWorks2012;

GO

/*

This section of the code joins the Person table with the Address table, by using the Employee and BusinessEntityAddress tables in the middle to get a list of all the employees in the AdventureWorks2012 database and their contact information.

*/

```
SELECT p.FirstName, p.LastName, a.AddressLine1,
a.AddressLine2, a.City, a.PostalCode
FROM Person.Person AS p
JOIN HumanResources.Employee AS e ON
p.BusinessEntityID = e.BusinessEntityID
JOIN Person.BusinessEntityAddress AS ea ON
e.BusinessEntityID = ea.BusinessEntityID
JOIN Person.Address AS a ON ea.AddressID =
a.AddressID;
GO
```

- Multi-line comment must be within one batch, it cannot span over two batch
- The following is invalid

/*

Comment Line -1

GO

Comment Line-2

*/

Variables in T-SQL

- A Transact-SQL variable is an object that can hold a single data value of a specific type.
- Variables in scripts are typically used:
 - As a counter either to count the number of times a loop is performed or to control how many times the loop is performed.
 - To hold a data value to be tested by a control-of-flow statement.
 - To save a data value to be returned by a stored procedure return code or function return value.

i

The names of some Transact-SQL system functions begin with two at signs (@@). Although in earlier versions of Microsoft SQL Server, the @@functions are referred to as global variables, they are not variables and do not have the same behaviors as variables. The @@functions are system functions, and their syntax usage follows the rules for functions

Variable Types

- Two types
 - Scalar variable: holds a single, atomic value
 - Table variable: holds a number of rows

Declaring scalar variables

- Variables are declared with the DECLARE statement.
- Syntax
DECLARE @variable DATA-TYPE
OR
DECLARE @variable AS DATA-TYPE
OR
DECLARE @variable1 data-type, variable2 DATA-TYPE
OR
DECLARE @variable DATA-TYPE = value
- Variable name must begin with @
- data-type any supported data type except text, ntext, or image
- A variable can be assigned a value in-line. The value can be a constant or an expression, but it must

either match the variable declaration type or be implicitly convertible to that type.

- Multiple variables can be defined using a single DECLARE statement.
DECLARE @i INT, @n NVARCHAR (30) = 'Dhaka'

Declaring table variables

- Almost same as scalars
DECLARE @variable-name TABLE
(
 column-definition, ...
)
- Example
DECLARE @custs TABLE (
 customerID CHAR(8),
 companyName VARCHAR(40)
)

Setting the Value in Your Variables

- Variables are assigned values by using either a SET or SELECT statement.

Setting Variables Using SET

- Syntax
SET @variable = value
→ The SET statements sets the specified variable to the specified value.
→ After a variable is declared, it is initialized to NULL. You can use the SET statement to assign a value to a declared variable.
→ The SET statement assigns a value to a single variable.
→ When you initialize multiple variables, use a separate SET statement for each local variable.
- Examples
- Printing the value of a variable initialized by using SET
DECLARE @myvar char(20);
SET @myvar = 'This is a test';
SELECT @myvar;
GO
- Using a compound assignment for a local variable
DECLARE @NewBalance int = 10;
SET @NewBalance *= 10;
SELECT @NewBalance;
GO
- Assigning a value from a query
USE Northwind;
GO
DECLARE @rows int;
SET @rows = (SELECT COUNT(*) FROM Customers);
SELECT @rows;
GO

Setting Variables Using SELECT

- SELECT statement can be used to assign a value that comes from a query.
- But you can SELECT as the same way as the SET statement.
- SELECT @variable is typically used to assign a single value into the variable.
- However, when expression is the name of a column, it can return multiple values. If the SELECT statement returns more than one value, the variable is assigned the last value that is returned.
- If the SELECT statement returns no rows, the variable retains its present value.
- One SELECT statement can initialize multiple local variables.
- Examples

- Using SELECT @variable to return a single value
USE Northwind
GO
DECLARE @var1 nvarchar(60);
SELECT @var1 = 'Generic Name';
SELECT @var1 = CompanyName
FROM Customers
WHERE CustomerID = 'ALFKI' ;
SELECT @var1 AS 'Company Name';
GO
- Using SELECT = aggregate value
USE Northwind
GO
DECLARE @c INT
SELECT @c = COUNT(*) FROM Customers
GO
- Setting values to multiple variables
USE Northwind
GO
DECLARE @f NVARCHAR(25), @c NVARCHAR(50)
SELECT @c = Country, @f = fax FROM Customers
WHERE CustomerID = 'ALFKI'
GO

Rules to use SET or SELECT goes like this:

- Use SET when you are performing a simple assignment of a variable—where your value is already known in the form of an explicit value or some other variable.
- Use SELECT when you are basing the assignment of your variable on a query.

System functions in SQL Server 2012

- SQL Server provides a number of system functions.
- They perform operations and return information about values, objects, and settings in an instance of SQL Server.
- Each system function name starts with @@
- There are over 30 parameterless system functions available.
- Most important of them are listed below:

Variable	Purpose	Comment
@@DATEFIRST	Returns what currently set as the first day of the week	Set in system wide
@@ERROR	Returns the error number for the last Transact-SQL statement executed.	Because @@ERROR is cleared and reset on each statement executed, check it immediately following the statement being verified, or save it to a local variable that can be checked later.
@@IDENTITY	Returns the last-inserted identity value.	@@IDENTITY contains the last identity value that is generated by the statement. If the statement did not affect any tables with identity columns, @@IDENTITY returns NULL. If multiple rows are inserted,

		generating multiple identity values, @@IDENTITY returns the last identity value generated.
@@ROWCOUNT	Returns the number of rows affected by the last statement	Commonly used in non-runtime error checking. For example, if you try to DELETE a row using a WHERE clause, and no rows are affected, then that would imply that something unexpected happened. You can then raise an error manually.

SQL Batches

- A batch is a grouping of T-SQL statements into one logical unit.
- SQL Server compiles the statements of a batch into a single executable unit, called an execution plan. The statements in the execution plan are then executed one at a time.
- Because a batch is compiled into a single execution plan, a batch must be logically complete.
- The execution plan created for one batch has no ability to reference any variables declared in another batch.
- Comments must both start and end in one batch.

Errors in batches

- Errors in batches fall into two categories:
 - Syntax errors
 - Runtime errors
- A compile error, such as a syntax error, prevents the compilation of the execution plan. Therefore, no statements in the batch are executed.
- A run-time error, such as an arithmetic overflow or a constraint violation, has one of the following effects:
 - Most run-time errors stop the current statement and the statements that follow it in the batch.
 - Some run-time errors, such as constraint violations, stop only the current statement. All the remaining statements in the batch are executed.
 - The statements that are executed before the statement that encountered the run-time error are not affected.
 - For example, assume there are 10 statements in a batch. If the fifth statement has a syntax error, no statements in the batch are executed. If the batch is compiled and the second statement then fails while running, the results of the first statement are not affected because it has already executed.

Rules for Using Batches

- The following rules apply to using batches:
 - CREATE DEFAULT, CREATE FUNCTION, CREATE PROCEDURE, CREATE RULE, CREATE SCHEMA, CREATE TRIGGER, and CREATE VIEW statements cannot be combined with other statements in a batch.
 - The CREATE statement must start the batch. All other statements that follow in that batch will be interpreted as part of the definition of the first CREATE statement.

- A table cannot be changed and adding new columns or altering a column cannot be done in the same batch.
- If an EXECUTE statement is the first statement in a batch, the EXECUTE keyword is not required. The EXECUTE keyword is required if the EXECUTE statement is not the first statement in the batch.

Using GO to separate batches

- GO Statement Signals the end of a batch of Transact-SQL statements to the SQL Server utilities.
- GO is not a Transact-SQL statement; it is a command recognized by the SQLCMD and OSQL utilities and SQL Server Management Studio Code editor.
- SQL Server utilities interpret GO as a signal that they should send the current batch of Transact-SQL statements to an instance of SQL Server.
- A Transact-SQL statement cannot occupy the same line as a GO command. However, the line can contain comments.

SQLCMD Utility

- The SQLCMD utility lets run Transact-SQL statements, and script files at the command prompt.
- You can also run T-SQL statements in SQLCMD mode in SQL Server Management Studio's query editor.
- In the SQLCMD environment, you can interactively enter Transact-SQL statements the same way you enter them when you are working at the command prompt. The results are displayed in the Command Prompt window.
- You can also run T-SQL statements stored in a file.
- You can send result sets to output files.

Working with SQLCMD

- To work with SQLCMD, you have run SQLCMD and connect with an SQL Server Instance.
- To SQLCMD run you have to specify some switches or options
- Each switch is prefixed with a -, and a space is required to separate two switches.
- Syntax

sqlcmd

- a packet_size
- A (dedicated administrator connection)
- b (terminate batch job if there is an error)
- c batch_terminator
- C (trust the server certificate)
- d db_name
- e (echo input)
- E (use trusted connection)
- f codepage [i:codepage[o:codepage] | o:codepage[i:codepage]
- h rows_per_header
- H workstation_name
- i input_file
- I (enable quoted identifiers)
- k[1 | 2] (remove or replace control characters)
- K application_intent
- l login_timeout
- L[c] (list servers, optional clean output)
- m error_level
- M multisubnet_failover
- N (encrypt connection)
- o output_file
- p[1] (print statistics, optional colon format)
- P password

-q "cmdline query"
 -Q "cmdline query" (and exit)
 -r [0 | 1] (msgs to stderr)
 -R (use client regional settings)
 -s col_separator
 -S [protocol:]server[instance_name][,port]
 -t query_timeout
 -u (unicode output file)
 -U login_id
 -v var = "value"
 -V error_severity_level
 -w column_width
 -W (remove trailing spaces)
 -x (disable variable substitution)
 -X[1] (disable commands, startup script, environment variables and optional exit)
 -y variable_length_type_display_width
 -Y fixed_length_type_display_width
 -z new_password
 -Z new_password (and exit)
 -? (usage)

- You don't have to remember these all. If you forget just run the following at the command window.
 SQLCMD - ?
- To begin using SQLCMD, you must first launch the utility and connect to an instance of SQL Server.
- You can connect to either a default or named instance. The first step is to start the SQLCMD utility.

Workout #01

- Starting SQLCMD

→ Starting the SQLCMD utility and connect to a named instance of SQL Server using windows authentication

- On the Start menu click Run.
- In the Open box type cmd, and then click OK to open a Command Prompt window.
- At the command prompt, type
 SQLCMD -S.\SQLEXPRESS -E
- Press ENTER.

→ You now have a trusted connection to the default instance of SQL Server that is running on your computer.

1> is the SQLCMD prompt that specifies the line number. Each time you press ENTER, the number increases by one.

- To end the SQLCMD session, type EXIT at the SQLCMD prompt.

→ Starting the SQLCMD utility and connect to a named instance of SQL Server using SQL Server authentication

- At the command prompt, type
 SQLCMD -S.\SQLEXPRESS -E -u Northwind -P Open123
- Press ENTER.

Workout #02

→ Running a single query in SQLCMD utility

- At the command prompt, type
 SQLCMD -S.\SQLEXPRESS -E -d Northwind -Q "SELECT TOP 5 ProductID, UnitPrice FROM Products ORDER BY UnitPrice"
- Press ENTER.

- Output

ProductID	UnitPrice
33	2.5000
24	4.5000
13	6.0000
52	7.0000
54	7.4500

(5 rows affected)

Workout #03

→ Running T-SQL statements interactively in SQLCMD

- At the command prompt, type
 SQLCMD -S.\SQLEXPRESS -E
- Press ENTER.
- Type in SQLCMD prompt
 1> USE Northwind
 2> GO

- Output

Changed database context to 'Northwind'.

- Type in SQLCMD prompt
 1> SELECT COUNT(*) FROM Customers
 2> GO

- Output

91

(1 rows affected)

- Type in SQLCMD prompt
 1> CREATE DATABASE testdb
 2> go
 1> USE testdb
 2> GO

- Output

Changed database context to 'testdb'

- Type in SQLCMD prompt
 1> CREATE TABLE tblTest
 2> (
 3> C1 INT IDENTITY NOT NULL,
 4> C2 NVARCHAR(20) NOT NULL
 5>)
 6> GO
 1> INSERT INTO tbltest
 2> (C2)
 3> VALUES ('Barte')
 4> INSERT INTO tbltest
 5> (C2)
 6> VALUES ('Forte')
 7> GO

- Output

(1 rows affected)

- SELECT * FROM tbltest
 2> GO

- Output

C1	C2
1	Barte
2	Forte

(2 rows affected)

- Type in SQLCMD prompt
 1> use master
 2> GO

- Output

Changed database context to 'master'.

- Type in SQLCMD prompt
 1> DROP DATABASE testdb
 2> GO
- Exit from SQLCMD

Workout #05

→ Running Transact-SQL Script Files by Using SQLCMD

- Open a new Query Window
- Type the following
 USE master
 GO

```

CREATE DATABASE testdb
GO
USE testdb
GO
CREATE TABLE tbltest
(
    C1 INT IDENTITY PRIMARY KEY,
    C2 NVARCHAR(30) NOT NULL
)
GO
DECLARE @lastid int
INSERT INTO tbltest (C2) VALUES ('Barte')
INSERT INTO tbltest (C2) VALUES ('Forte')
INSERT INTO tbltest (C2) VALUES ('Dambol')
SELECT @lastid = @@IDENTITY
SELECT @lastid AS 'Last identity value inserted'
GO
SELECT * FROM tbltest
GO
USE master
GO
DROP DATABASE testdb
GO

```

3. Save the file sqlscript.sql in drive D:
 4. Open a command prompt window.
 5. In the Command Prompt window, type:
SQLCMD -S.\SQLEXPRESS -E -i "D:\sqlscript.sql"
-o "D:\output.txt"
 6. Press ENTER.
- No output is returned in the Command Prompt window. Instead, the output is sent to the EmpAdds.txt file. You can verify this output by opening the EmpAdds.txt file.

Executing Dynamic SQL with EXEC Command

- EXEC command allows you to build our SQL statement on-the-fly using string manipulation and execute it.
- Syntax
EXEC (<string variable>| '<literal command string>')
- Or:
EXECUTE (<string variable>| '<literal command string>')

Workout #06

- Try this
DECLARE @sqlCommand varchar(1000)
DECLARE @columnList varchar(75)
DECLARE @city varchar(75)
SET @columnList = 'CustomerID, ContactName, City'
SET @city = 'London'
SET @sqlCommand = 'SELECT ' + @columnList +
' FROM customers WHERE City = ' + @city
EXEC (@sqlCommand)
GO
- Output

CustomerID	ContactName	City
AROUT	Thomas Hardy	London
BSBEV	Victoria Ashworth	London
CONSH	Elizabeth Brown	London
EASTC	Ann Devon	London
NORTS	Simon Crowther	London
SEVES	Hari Kumar	London

Workout #07

- Try this
USE Northwind
GO
CREATE PROC Search @strids VARCHAR(100)
AS
DECLARE @SQL VARCHAR (1000)

```

SELECT @SQL = 'SELECT ProductName, UnitPrice,
UnitsInStock FROM Products '
SELECT @SQL = @SQL + 'WHERE ProductID in (' +
@strids + ')'

```

```

EXEC ( @SQL)
GO
EXEC Search '1, 3, 5, 6'
GO

```

Output

ProductName	UnitPrice	UnitsInStock
Chai	18.00	39
Aniseed Syrup	10.00	13
Chef Anton's Gumbo Mix	21.35	0
Grandma's Boysenberry Spread	25.00	120

Restrictions in EXEC command

- It runs under a separate scope than the code that calls it—that is, the calling code can't reference variables inside the EXEC statement, and the EXEC can't reference variables in the calling code after they are resolved into the string for the EXEC statement.
- It runs under the same security context as the current user—not that of the calling object.
- It runs under the same connection and transaction context as the calling object.
- Concatenation that requires a function call must be performed on the EXEC string prior to actually calling the EXEC statement—you can't do the concatenation of function in the same statement as the EXEC call.
- EXEC cannot be used inside a User Defined Function.

Practice #01

- You have to create a SQL Script batch file that has SQL statements performing the following tasks
1. Creates a database named "EVIDENCE"
 2. Creates a table in EVIDENCE database like below

Table: tblTest

Column name	Data Type
C1	VARCHAR(50)
C2	INT

3. Inserts the following data into the table

C1	C2
ABC	10
PQR	20
XYZ	30

4. Deletes the database
- Now do the following tasks
- A. Save SQL statements in a file
 - B. RUN the script file using SQLCMD utility and store the generated output in "output.txt" file